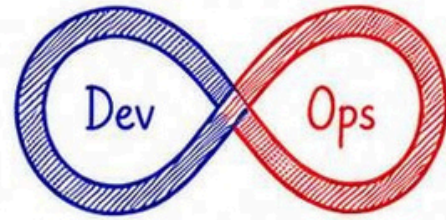


What is DevOps?



DevOps is a culture and set of practices that combines development and operations to deliver software faster and more reliably.



★ Key Goals of DevOps

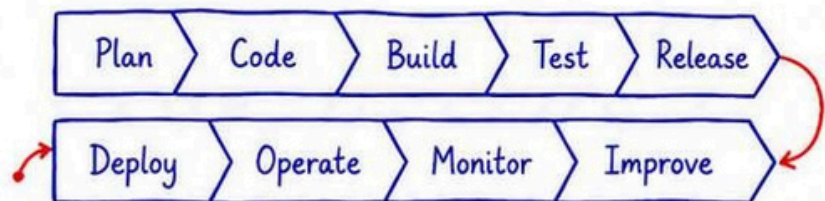
- Automation
- Faster delivery
- Improved collaboration
- Reliability
- Scalability
- Continuous feedback



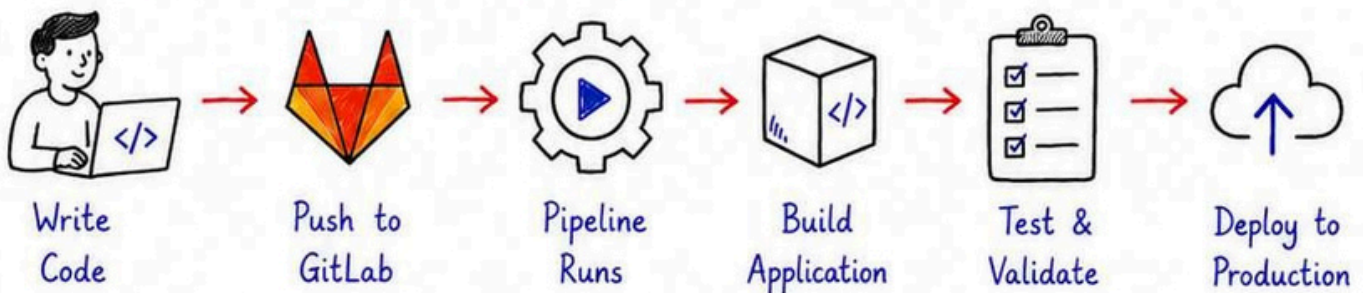
Assma Fadhli



★ DevOps Lifecycle



★ How It All Works Together



★ Benefits



Faster Delivery



Better Collaboration



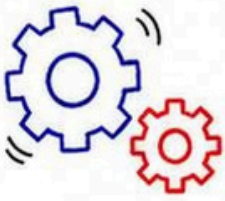
Higher Reliability



Improved Scalability



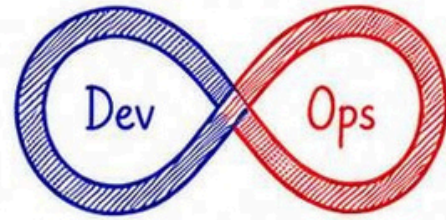
Continuous Feedback



What is DevOps?



DevOps is a culture and set of practices that combines development and operations to deliver software faster and more reliably.



★ Key Goals of DevOps

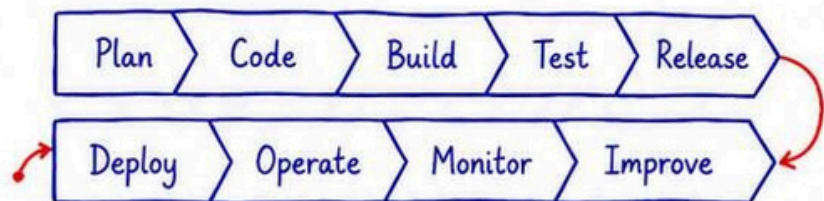
- Automation
- Faster delivery
- Improved collaboration
- Reliability
- Scalability
- Continuous feedback



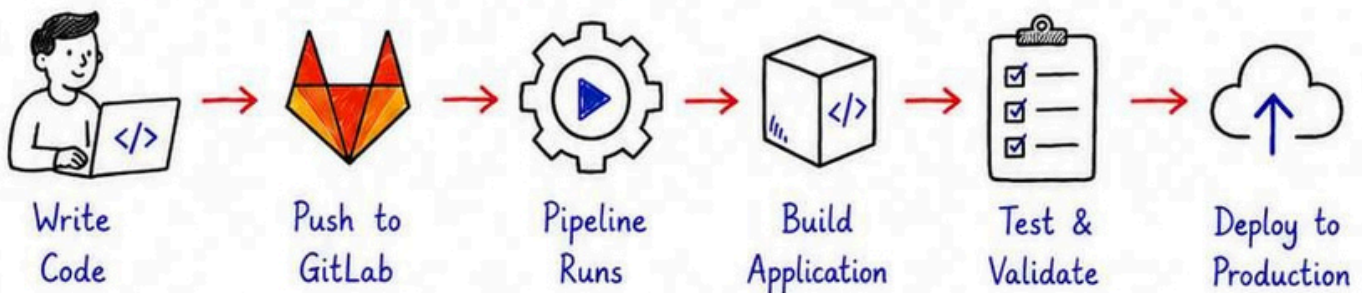
Assma Fadhli



★ DevOps Lifecycle



★ How It All Works Together



★ Benefits



Faster Delivery



Better Collaboration



Higher Reliability



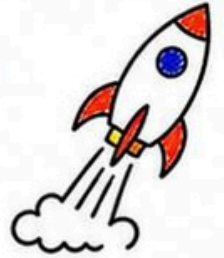
Improved Scalability



Continuous Feedback

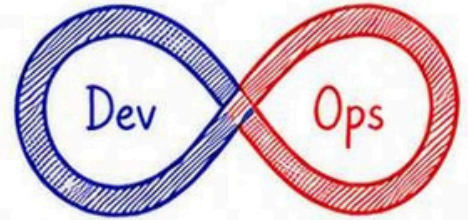


DevOps in Action



What is DevOps in Real World?

- DevOps connects Development and Operations teams.
- It automates the process of building, testing, and deploying code.
- This helps deliver software faster, safer, and with fewer errors.

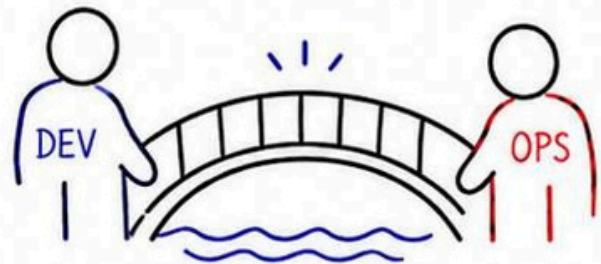


★ Why DevOps?

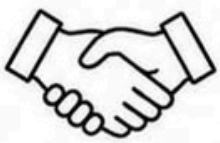
Assma Fadhli



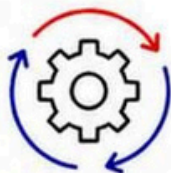
- Bridge the gap between Dev and Ops
- Faster time to market
- High quality and reliable releases
- Continuous improvement
- Better collaboration and visibility



★ DevOps Principles



Collaboration
& Communication



Automation
Everywhere



Continuous
Feedback



Security
as Code



Measure
& Improve

★ DevOps Culture



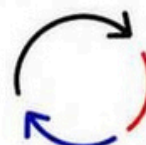
Ownership



Trust &
Respect



Learning
Mindset



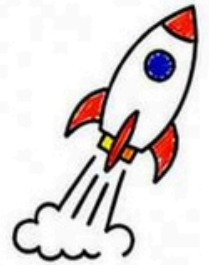
Embrace
Change



Transparency

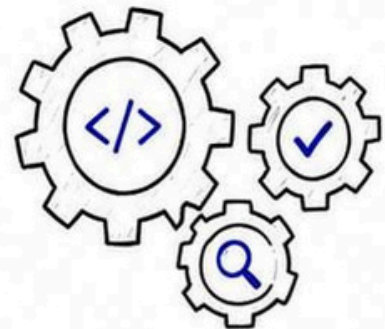
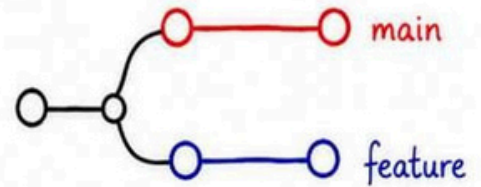


CI/CD in GitLab



What is CI/CD?

- CI = Continuous Integration
- CD = Continuous Delivery / Continuous Deployment
- GitLab CI/CD automatically builds, tests, and deploys an application whenever code is pushed to a GitLab repository.



★ Continuous Integration

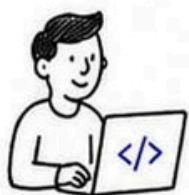
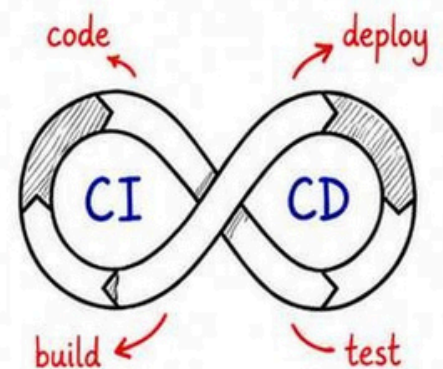
- Developers frequently push code to a shared repository
- GitLab automatically builds the code
- It runs tests and quality checks
- Problems are found early before merge

Assma Fadhli

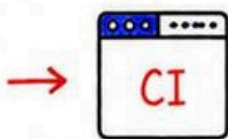


✓ Continuous Delivery / Deployment

- After successful testing, the app is prepared for release
- Delivery usually needs manual approval
- Deployment can release automatically after all checks pass



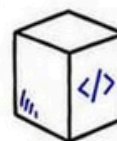
Developer
writes code



Pushes to
GitLab



Pipeline
starts



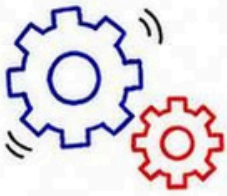
Build



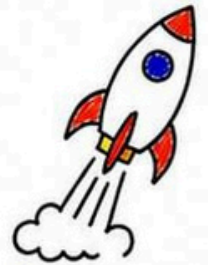
Test



Deploy

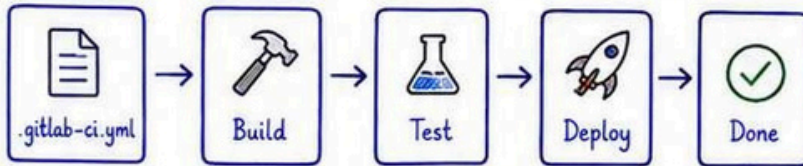


CI/CD in GitLab



CI/CD Pipeline Stages

A pipeline is made of ordered stages.
Jobs within a stage run in parallel.



Example: .gitlab-ci.yml

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Building the app"

test_job:
  stage: test
  script:
    - echo "Running tests"

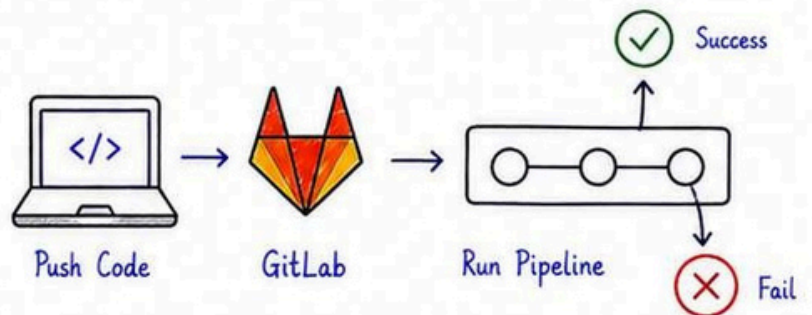
deploy_job:
  stage: deploy
  script:
    - echo "Deploying the app"
```

Assma Fadhli



How a Pipeline Runs

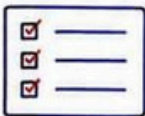
- 1 Developer pushes code to GitLab
- 2 GitLab triggers the pipeline
- 3 Jobs run by stages (as defined in .gitlab-ci.yml)
- 4 If all jobs succeed, the code is ready to deploy
- 5 If a job fails, the pipeline stops (unless configured otherwise)



Jobs, Stages & Dependencies

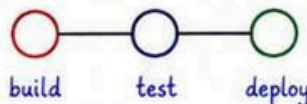
Jobs

A job is a task in the pipeline.
Example: build, test, lint.



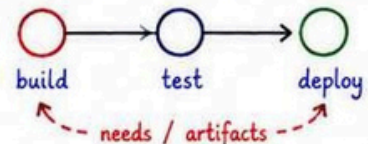
Stages

Stages define the order.
Default: build → test → deploy.



Dependencies

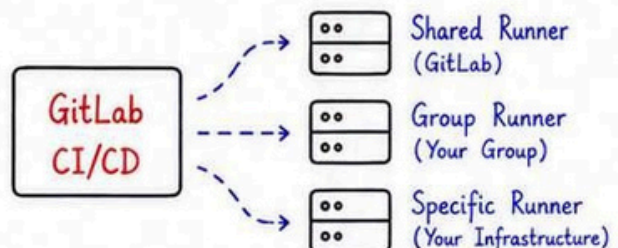
Jobs can depend on other jobs using needs or artifacts.

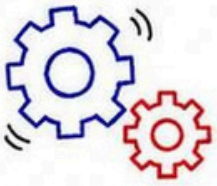


Runners

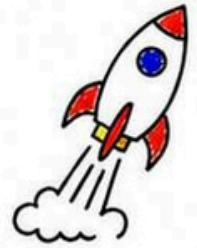
Runners are agents that run jobs.
They can be:

- **Shared Runners** (GitLab provides)
- **Group Runners** (for your group)
- **Specific Runners** (your own servers or Docker, Kubernetes, etc.)





CI/CD in GitLab



.gitlab-ci.yml Structure

The CI/CD configuration file lives at the root of your project.

It defines:

- Stages (order of execution)
- Jobs (tasks to run)
- Scripts (commands)
- Rules/Conditions
- Dependencies between jobs

Assma Fadhli



Minimal Example

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Building..."

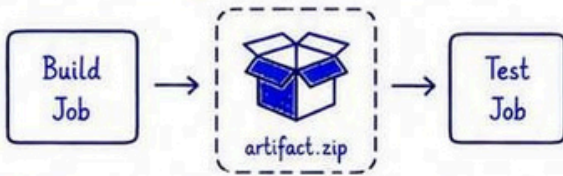
test_job:
  stage: test
  script:
    - echo "Testing..."

deploy_job:
  stage: deploy
  script:
    - echo "Deploying..."
```

Artifacts & Caching

★ Artifacts

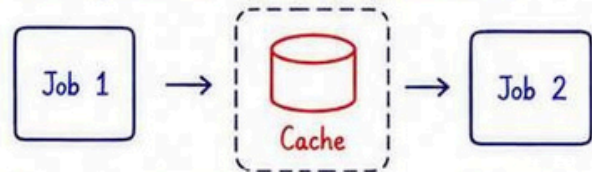
Artifacts are files generated by jobs and passed to later jobs or downloaded.



Useful for sharing build outputs, reports, binaries, etc.

Cache

Cache stores dependencies (like node_modules, .m2, .gradle) to speed up pipeline runs.



Reduces download time and improves performance.

Variables

Variables store values that can be used in jobs and scripts.

They can be:

- Predefined (GitLab provides)
- Custom (defined in UI, group, or project)
- Secret (masked & protected)

Variables Example

Key	Value	Type	Scope
DEPLOY_ENV	production	Variable	Protected
AWS_ACCESS_KEY_ID	*****	Masked	All
API_URL	https://api.example.com	Variable	All

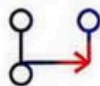
Pipeline Triggers

Push



Triggered when code is pushed to a branch.

Merge Request



Runs pipeline for MRs before merging.

Schedule



Run pipelines at specific time.

Manual

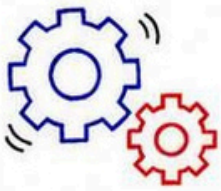


Triggered manually from GitLab UI.

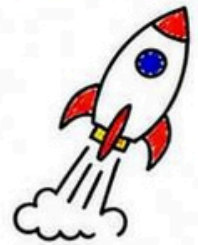
API Trigger



Triggered using GitLab API or external tools.



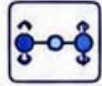
CI/CD in GitLab



GitLab CI/CD Components



.gitlab-ci.yml : Defines the pipeline configuration.



Pipeline : Collection of stages and jobs executed in order.



Job : A task that runs a script in a runner.



Script : Commands executed by the job.

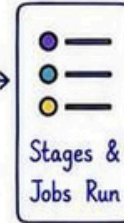
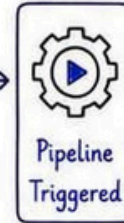
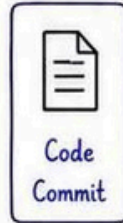


Artifact : Files generated by a job and shared.



Cache : Files reused between pipeline runs.

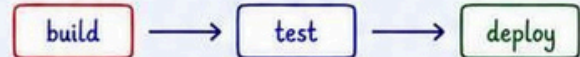
Pipeline Flow



Stage Execution Order

Stages run in the order defined.

Jobs in the same stage run in parallel.



Next stage starts only if previous stage succeeds.

Job Statuses



success

Job completed successfully.



failed

Job failed.



running

Job is currently running.



pending

Job is waiting to start.



skipped

Job was skipped (by rules or conditions).



manual

Job requires manual intervention.

Rules & Conditions

Control when jobs or pipelines run using rules or only/except.

```
job_example:
  script: echo "Hello"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always
    - when: never
```

Assma Fadhli



Run only on main branch

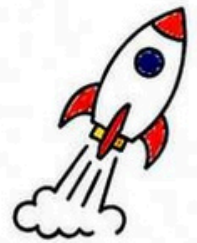
Otherwise, don't run

Artifacts vs Cache

	Artifacts	Cache
Purpose	Share files between jobs or for download after pipeline.	Speed up pipeline by reusing dependencies.
Storage	Stored as part of pipeline.	Stored separately and reused across pipelines.
Download	Manually or by dependent jobs.	Automatically restored.
Retention	Expires after a defined time.	Can be kept for longer and reused.
Example	Build binaries, reports, test results	npm packages, pip cache, Maven repo



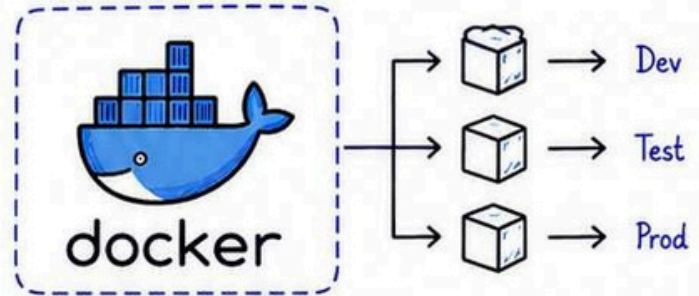
Docker in DevOps



What is Docker?

Docker is a platform that allows you to build, package, and run applications in lightweight containers.

It ensures consistency across different environments.



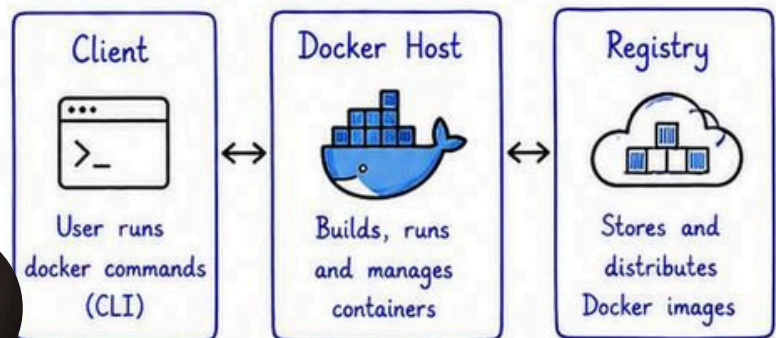
★ Why Use Docker?

- Consistent environments from dev to prod
- Lightweight and fast
- Easy to scale and deploy
- Isolated applications
- Improves collaboration

Assma Fadhli



Docker Architecture



Key Docker Concepts



Image : A read-only template used to create containers.



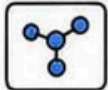
Container : A running instance of an image.



Dockerfile : A file with instructions to build an image.



Volume : Persistent data storage for containers.



Network : Enables communication between containers.

Common Docker Commands

```
docker build -t myapp .
```

Build an image from a Dockerfile.

```
docker run -d -p 8080:80 myapp
```

Run a container in detached mode.

```
docker ps
```

List running containers.

```
docker images
```

List available images.

```
docker stop <container_id>
```

Stop a running container.

```
docker rm <container_id>
```

Remove a stopped container.

```
docker logs <container_id>
```

View logs of a container.

Typical Docker Workflow



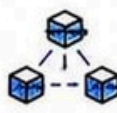
Write Dockerfile



Build Image



Run Container



Deploy & Scale



Monitor & Manage

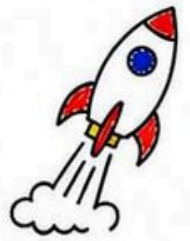


Tip

Keep your images small by using `.dockerignore` and lightweight base images like alpine.



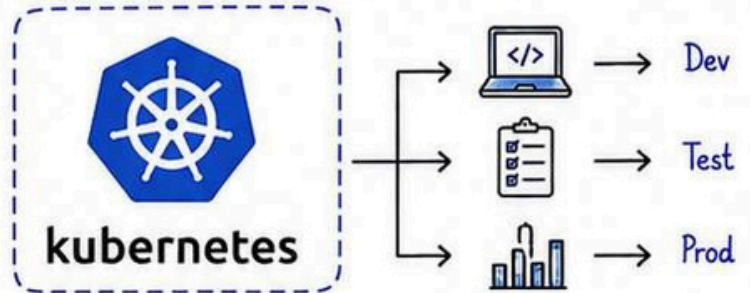
Kubernetes in DevOps



What is Kubernetes?

Kubernetes (K8s) is an open-source platform that automates the deployment, scaling, and management of containerized applications.

It helps you run your apps reliably at scale.



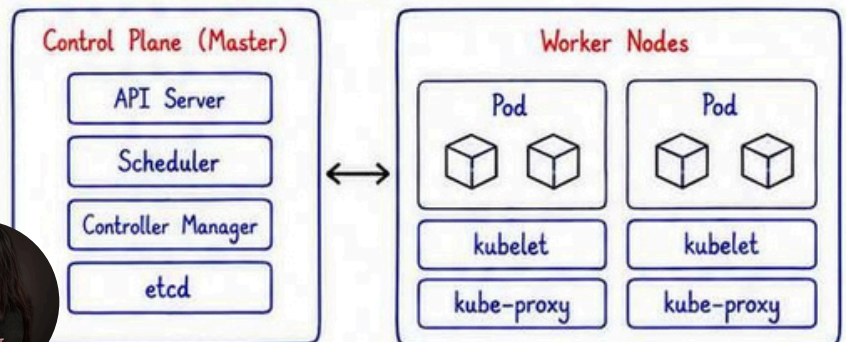
★ Why Use Kubernetes?

- Automates deployment and scaling
- High availability and self-healing
- Efficient resource utilization
- Easy rollouts and rollbacks
- Works across clouds and on-premises

Assma Fadhli



Kubernetes Architecture



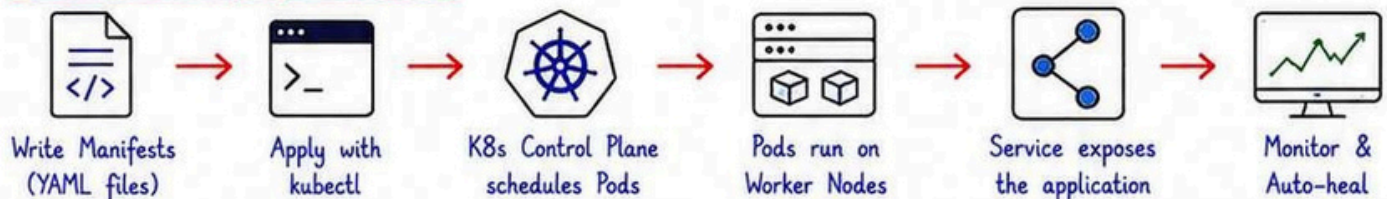
Key Kubernetes Concepts

- Pod** : Smallest deployable unit, one or more containers.
- Deployment** : Manages replicas of Pods.
- Service** : Stable network endpoint to access Pods.
- Namespace** : Virtual cluster inside a cluster.
- ConfigMap & Secret** : Store configuration and sensitive data.

Common kubectl Commands

<code>kubectl get pods</code>	List all pods.
<code>kubectl get svc</code>	List all services.
<code>kubectl get nodes</code>	List all nodes.
<code>kubectl describe pod <pod-name></code>	Show details about a pod.
<code>kubectl logs <pod-name></code>	View logs of a pod.
<code>kubectl apply -f <file.yaml></code>	Create or update resources.
<code>kubectl delete -f <file.yaml></code>	Delete resources.
<code>kubectl scale deployment <name> --replicas=3</code>	Scale a deployment.

Kubernetes Deployment Flow



Kubernetes Resources (Types)



Pod



Deployment



ReplicaSet



Service



Ingress



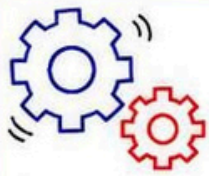
ConfigMap



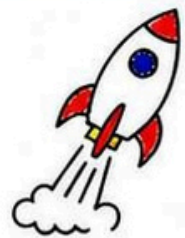
Secret



PersistentVolume

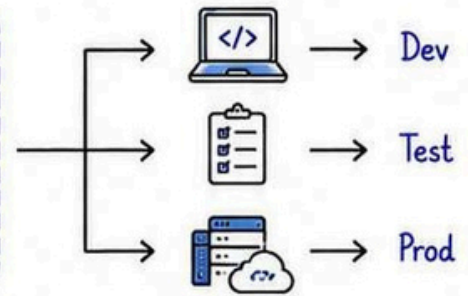


≡ Terraform in DevOps ≡



What is Terraform ?

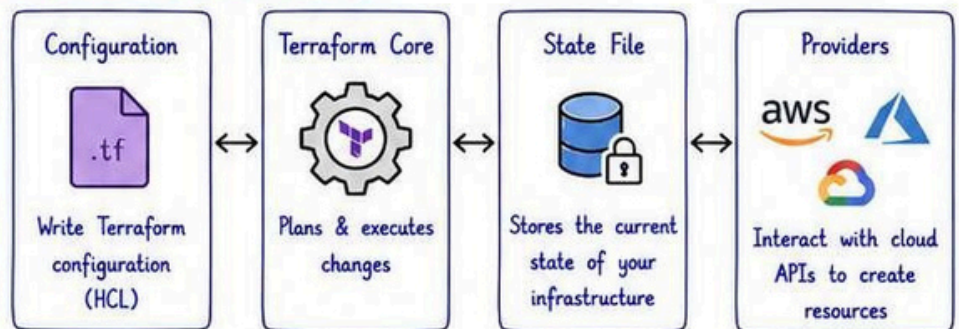
Terraform is an Infrastructure as Code (IaC) tool that lets you define, provision, and manage cloud resources using code. It works with multiple cloud providers.



★ Why Use Terraform ?

- Infrastructure as Code
- Consistent & Reproducible
- Version controlled
- Multi-cloud & Hybrid support
- Plan before you apply (safety)
- Collaborative & extensible

Terraform Architecture



Key Terraform Concepts

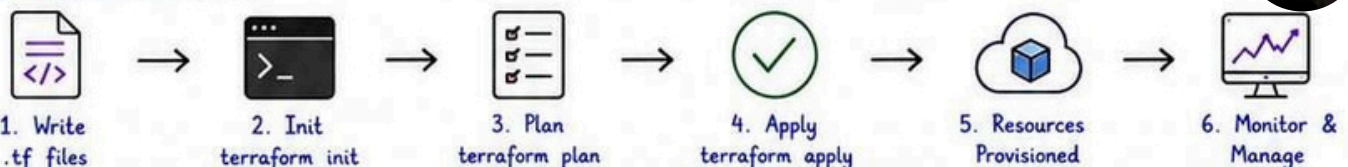
- Resource** : A manageable infrastructure object (VM, S3 bucket, VPC...).
- Provider** : A plugin that allows Terraform to interact with APIs.
- Variable** : Makes configurations dynamic and reusable.
- Module** : Reusable set of Terraform configurations.
- State** : Maps real infrastructure to your Terraform configuration.

Common Terraform Commands

- | | |
|---------------------------------|---|
| <code>terraform init</code> | Initialize a Terraform working directory. |
| <code>terraform validate</code> | Validate the configuration files. |
| <code>terraform plan</code> | Preview the changes Terraform will make. |
| <code>terraform apply</code> | Apply the changes to reach the desired state. |
| <code>terraform destroy</code> | Destroy all resources managed by Terraform. |
| <code>terraform fmt</code> | Format configuration files. |
| <code>terraform output</code> | Show output values from the state. |

Terraform Workflow

Assma Fadhli



Example: Create an AWS EC2 Instance

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbf1f0"
  instance_type = "t2.micro"
  tags = {
    Name = "MyWebServer"
  }
}
```

terraform plan (output)

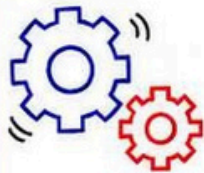
```
+ aws_instance.web
  ami           = "ami-0c55b159cbf1f0"
  instance_type = "t2.micro"
  tags.Name     = "MyWebServer"

Plan: 1 to add, 0 to change, 0 to destroy.
```



Tip

Always run terraform plan before apply to review changes and avoid surprises!



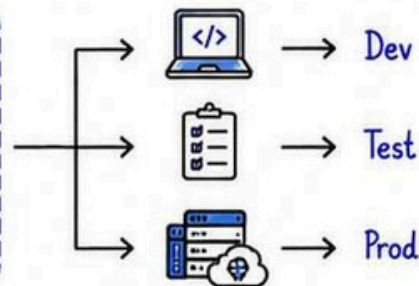
Ansible in DevOps



What is Ansible ?

Ansible is an open-source automation tool that helps you configure, deploy, and manage systems without installing any agent.

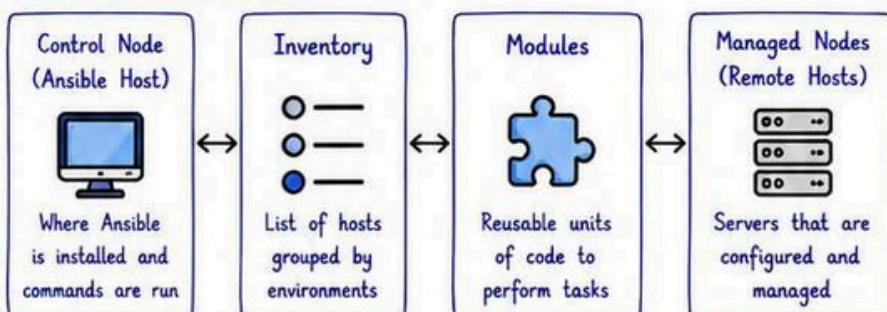
It uses SSH to connect to remote servers and YAML for writing automation.



★ Why Use Ansible ?

- Agentless (uses SSH)
- Easy to learn (YAML syntax)
- Idempotent (safe to run multiple times)
- Consistent configuration
- Huge community & modules
- Cross-platform (Linux, Windows, macOS)

Ansible Architecture



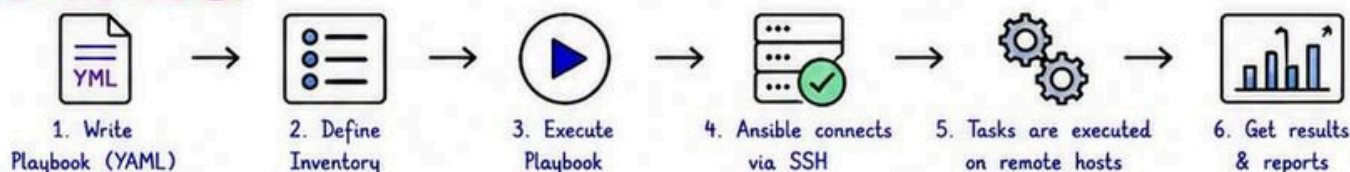
Key Ansible Concepts

- Inventory** : List of hosts or groups where tasks are executed.
- Playbook** : YAML file containing a set of tasks to achieve a goal.
- Module** : Reusable code units that Ansible executes on remote hosts.
- Role** : Organize playbooks, tasks, vars, and templates.
- Task** : A single step executed by Ansible.
- Handler** : Tasks triggered by changes.

Common Ansible Commands

- | | |
|---|----------------------------------|
| <code>ansible all -m ping</code> | Check connectivity to all hosts. |
| <code>ansible all -m command -a "uptime"</code> | Run a command on all hosts. |
| <code>ansible-playbook site.yml</code> | Run a playbook. |
| <code>ansible-galaxy install <role_name></code> | Install a role from Galaxy. |
| <code>ansible-vault create secrets.yml</code> | Create an encrypted file. |
| <code>ansible-vault edit secrets.yml</code> | Edit an encrypted file. |
| <code>ansible-doc <module_name></code> | Get documentation for a module. |

Ansible Workflow



Assma Fadhli



Example: Simple Playbook

```

---
- name: Install and start nginx
  hosts: webserver
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
        enabled: yes
  
```

Inventory Example (hosts.ini)

```

[webserver]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

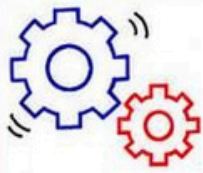
[dbserver]
db1 ansible_host=192.168.1.20
  
```



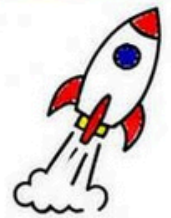
Tip

Use `--check` with playbooks to perform a dry run without making changes.

Example: `ansible-playbook site.yml --check`



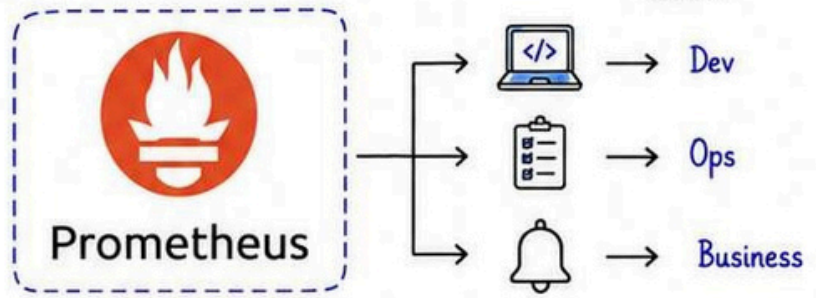
≡ Prometheus in DevOps ≡



What is Prometheus?

Prometheus is an open-source monitoring and alerting toolkit originally built by SoundCloud.

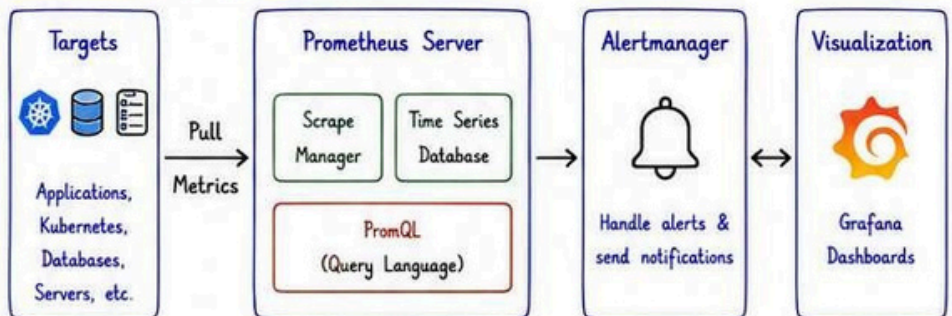
It collects metrics, stores them as time series data, and provides powerful querying and alerting capabilities.



★ Why Use Prometheus?

- Multi-dimensional data model (time series)
- Powerful query language (PromQL)
- Pull-based metrics collection
- Highly scalable & reliable
- Great integration with Grafana & Alertmanager
- Open-source and active community

Prometheus Architecture



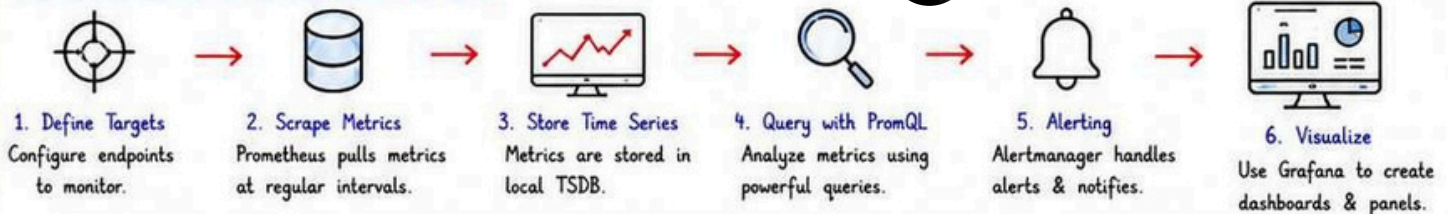
Key Prometheus Concepts

- Metric** : A single measurement (e.g., `cpu_usage`).
- Label** : Key-value pair attached to metrics (e.g., `instance="10.0.0.1"`).
- Time Series** : Set of metrics identified by metric name + labels.
- Sample** : A value of a metric at a specific timestamp.
- PromQL** : Query language to fetch and manipulate metrics.
- Alert** : Defined conditions that trigger notifications.

Common PromQL Queries

<code>up</code>	Returns 1 if the target is up, 0 if down.
<code>rate(http_requests_total[5m])</code>	Request rate per second over 5 minutes.
<code>sum by (instance) (cpu_usage)</code>	Sum CPU usage grouped by instance.
<code>avg_over_time(memory_usage[1h])</code>	Average memory usage over 1 hour.
<code>increase(http_requests_total[1h])</code>	Total increase in requests in 1 hour.
<code>histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))</code>	95th percentile of request duration (latency).

Prometheus Monitoring Workflow



Assma Fadhli



Example: Prometheus Scrape Config (prometheus.yml)

```

scrape_configs:
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['localhost:9100']

  - job_name: 'api-service'
    static_configs:
      - targets: ['api:8080']
  
```

Example: Alert Rule

```

groups:
  - name: cpu_alerts
    rules:
      - alert: HighCPUUsage
        expr: avg(rate(cpu_usage[5m])) > 0.8
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High CPU usage detected"
          description: "CPU usage is above 80% for 5 minutes."
  
```



Tip

Combine Prometheus with Grafana for powerful visualization and Alertmanager for reliable alerting.

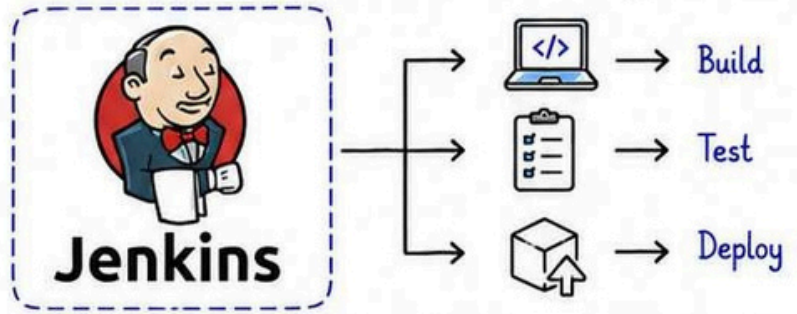


Jenkins in DevOps



What is Jenkins ?

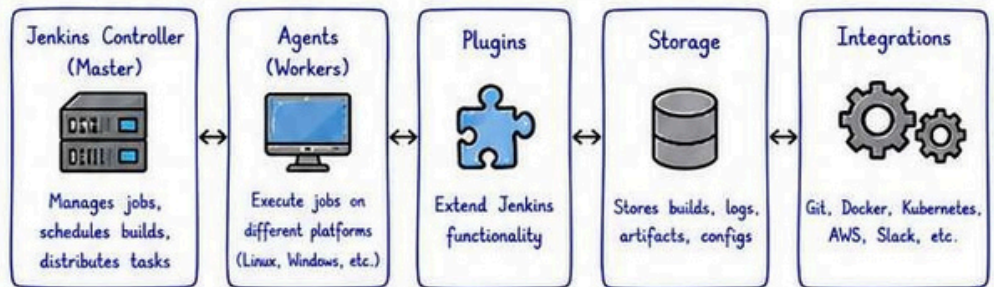
Jenkins is an open-source automation server that helps you build, test, and deploy your software continuously. It supports pipelines, plugins, and integrates with many DevOps tools.



★ Why Use Jenkins ?

- Automates the entire CI/CD pipeline
- Thousands of plugins & integrations
- Highly customizable
- Free and open-source
- Large community support

Jenkins Architecture



Key Jenkins Concepts

- Job** : A task or project that Jenkins builds and manages.
- Pipeline** : Defines a set of stages for CI/CD workflows.
- Build** : An execution of a job or pipeline.
- Plugin** : Adds new features and integrations.
- Node** : A machine (agent) that runs jobs.
- Artifact** : Output files generated by the build.

Common Jenkins Features

Feature	Description
Pipeline as Code	Define pipelines using Jenkinsfile (declarative or scripted).
SCM Integration	Integrates with Git, Bitbucket, GitLab, etc.
Build Triggers	Trigger builds on commit, schedule, webhook, or manual.
Test Reports	Publish and visualize unit test results (JUnit, TestNG...).
Notifications	Send notifications via Email, Slack, Microsoft Teams, etc.
Security	Role-based access control, credentials management.
Docker Integration	Build Docker images and push to registries.
Kubernetes Integration	Deploy applications to Kubernetes clusters.

Typical Jenkins Pipeline Flow



Assma Fadhli



Example: Jenkinsfile (Declarative Pipeline)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git 'https://github.com/user/repo.git' }
    }
    stage('Build') {
      steps { sh 'mvn clean package' }
    }
    stage('Test') {
      steps { sh 'mvn test' }
    }
    stage('Deploy') {
      steps { sh 'docker build -t myapp:latest .' }
    }
  }
}
```

Useful Jenkins CLI Commands

jenkins-cli list-jobs	List all jobs
jenkins-cli build <job_name>	Trigger a build
jenkins-cli console <build_no>	View build console logs
jenkins-cli node list	List all Jenkins nodes
jenkins-cli who-am-i	Show current user

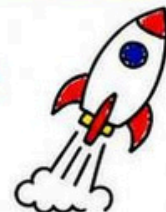


Tip

Store your pipeline as code (Jenkinsfile) in your repo. This makes your CI/CD process version controlled and reproducible!



DevOps Real-World Project



Project Overview (Part 1/3)

Project Scenario

We will build and deploy a CI/CD pipeline for a sample web application. The application is a simple Python Flask API containerized and deployed on AWS. All infrastructure is managed as code using Terraform, and monitoring & logging are integrated for observability and alerts.

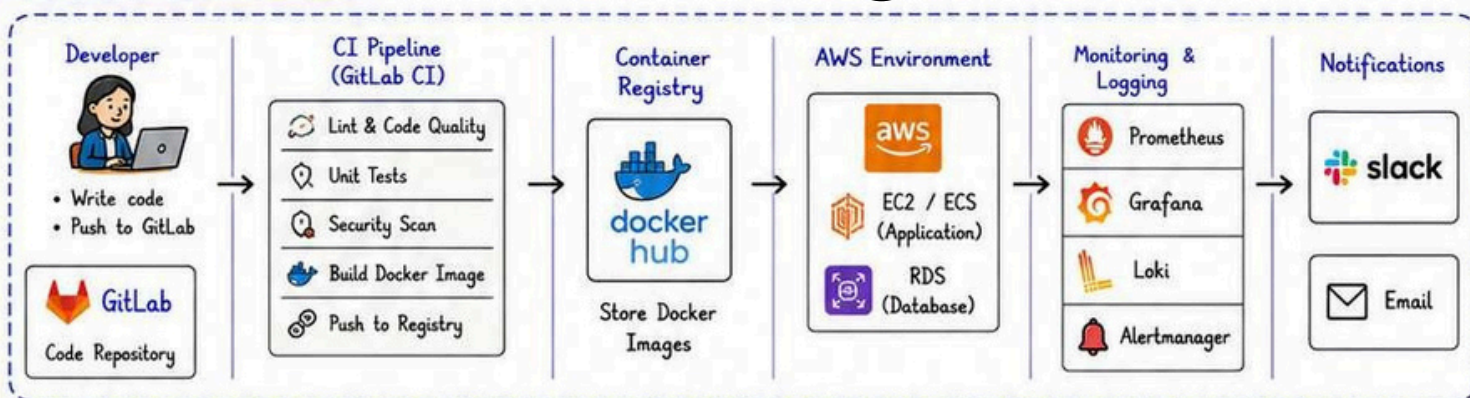


Our Goal

- ✓ Automate build, test and security checks
- ✓ Package the application
- ✓ Deploy to AWS environment
- ✓ Monitor application and infrastructure
- ✓ Get notified in case of failure
- ✓ Ensure everything is versioned and reproducible

High-Level Architecture

Assma Fadhli



Technologies Used

- GitLab** : Source code management and CI/CD orchestration.
- Docker** : Containerization of the application.
- AWS (EC2, RDS, S3, IAM)** : Cloud infrastructure to host the app.
- Terraform** : Infrastructure as Code (IaC) on AWS.
- Prometheus** : Metrics collection.
- Grafana** : Dashboards & visualization.
- Loki** : Log aggregation.
- Alertmanager** : Alert routing.
- Slack** : Notifications.

Project Components

Component	Description
Web App	Simple Flask API with /health endpoint.
CI/CD Pipeline	Automates test, build, scan and deploy.
Docker Image	Contains the application and its dependencies.
Infrastructure (AWS)	EC2 instance to run the app, RDS for database, S3 for artifacts, IAM for permissions.
Monitoring Stack	Prometheus + Grafana for metrics, Loki for logs, Alertmanager for alerts.
Notifications	Alerts sent to Slack channel and Email.

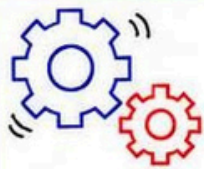
Repository Structure (Example)

```
my-flask-app/
├── app/
│   └── main.py
├── tests/
│   └── test_main.py
├── Dockerfile
├── requirements.txt
├── .gitlab-ci.yml
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
└── README.md
```

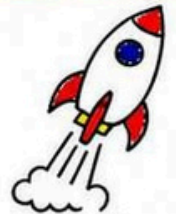
- Application source code
- Unit tests
- Docker build instructions
- Python dependencies
- CI/CD pipeline definition
- Terraform configuration for AWS resources
- Project documentation

What we'll build ?

- Code push to GitLab triggers the pipeline.
- Tests are executed.
- Docker image is built and pushed to Docker Hub.
- Infrastructure is deployed using Terraform.
- Application is deployed on AWS.
- Metrics and logs are collected and visualized.
- Alerts are sent if something goes wrong.



DevOps Real-World Project

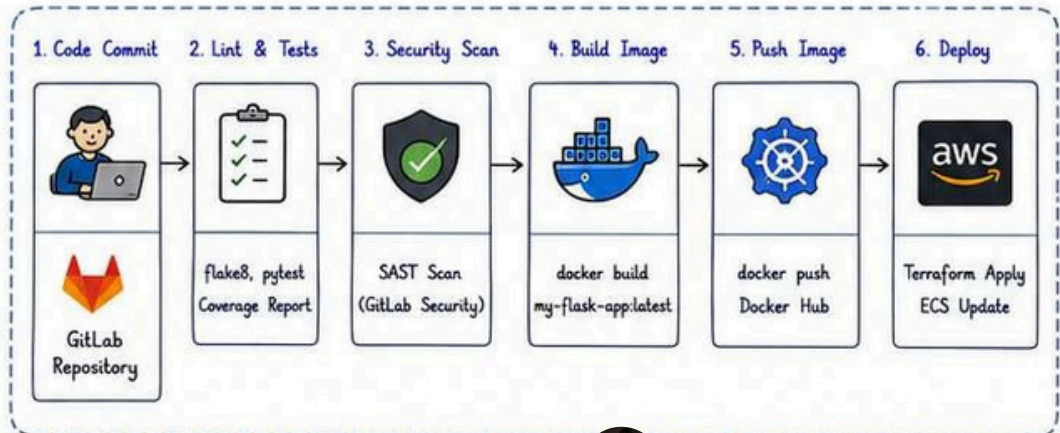


Project Walkthrough (Part 2/3)

CI/CD Pipeline Details

We use GitLab CI to automate the entire software delivery process.

1. **Code Commit**
Developer pushes code to GitLab.
2. **Lint & Unit Tests**
Code is linted and unit tests are executed.
3. **Security Scan**
Run SAST (static analysis) to detect vulnerabilities.
4. **Build Docker Image**
If all checks pass, build the Docker image.
5. **Push to Registry**
The image is pushed to Docker Hub.
6. **Deploy to AWS**
Terraform applies infrastructure and the new image is deployed to ECS.



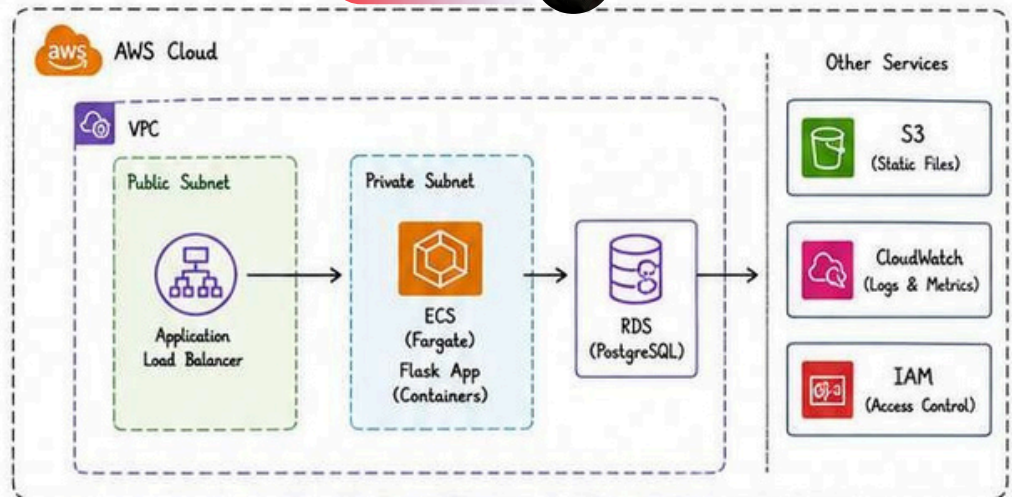
Assma Fadhli

Infrastructure Overview (AWS)

Our infrastructure is provisioned and managed as Code using Terraform.

Key Points

- ECS Fargate runs our containers without managing servers.
- RDS stores application data.
- ALB distributes traffic to application containers.
- S3 stores static files (if any).
- IAM roles are used for secure access.



Monitoring & Logging

Tool	What we monitor / collect
Prometheus	Collects metrics from the application and infrastructure (CPU, Memory, Requests...).
Grafana	Dashboards to visualize metrics and application performance.
Loki	Aggregates logs from containers and infrastructure.
Alertmanager	Handles alerts and sends notifications when rules are triggered.
Slack	Receive real-time alerts and important notifications.

Example Grafana Dashboard



Useful Commands

```
# Build and push Docker image
docker build -t your-dockerhub-username/my-flask-app:latest
docker push your-dockerhub-username/my-flask-app:latest

# Deploy infrastructure
cd terraform
terraform init
terraform plan
terraform apply

# Update ECS service with new image
aws ecs update-service --cluster my-cluster --service my-service \
--force-new-deployment
```



Key Takeaway

This project shows how DevOps practices and tools work together to deliver an application faster, safer and more reliably.

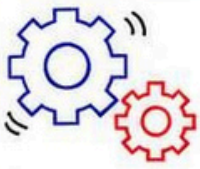
Automate everything, monitor everything, improve continuously!

Next Page (Part 3/3)

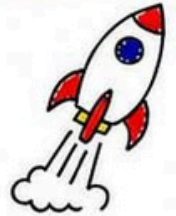
In the next page, we will see :

- Alerting rules examples
- Failure scenario & recovery
- Lessons learned & best practices





DevOps Real-World Project



Project Walkthrough (Part 3/3)

Deployment & Rollback

We use Terraform to manage infrastructure changes in AWS. Rollbacks are simple and safe.

✓ Deployment Flow

- 1 Terraform plan
Preview changes
- 2 Terraform apply
Apply infrastructure
- 3 New version of the app
deployed on ECS
- 4 Health checks pass
Traffic is served to the new tasks



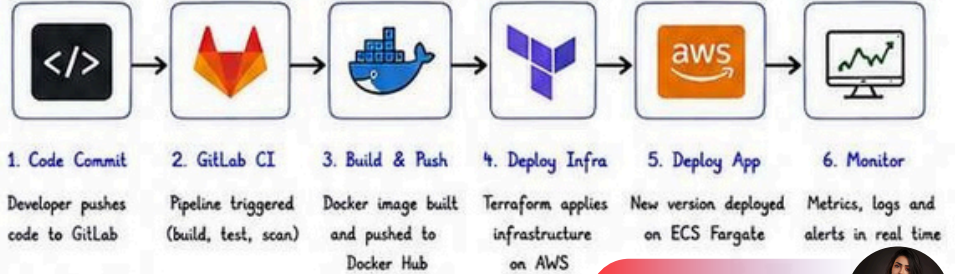
↻ Rollback Flow

- 1 Detect issue (alerts / user reports)
- 2 Rollback to previous task definition
or Docker image
- 3 Terraform apply to revert
infrastructure if needed
- 4 Service restored to a stable state

Testing & Quality Assurance

	Unit Tests	Pytest is used to test Python functions and endpoints.
	Security Scan	SAST scan with GitLab Security (Sengrep) to find vulnerabilities.
	Code Quality	Flake8 checks style and code quality.
	Coverage Report	Coverage is generated with pytest --cov and published in GitLab.
	Image Scan	Docker image is scanned for vulnerabilities before pushing.

End-to-End Flow

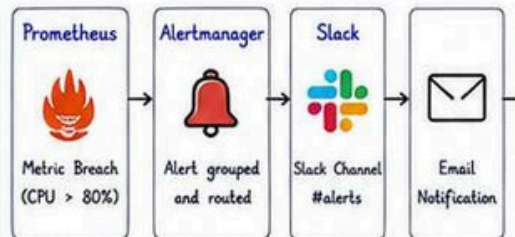


Assma Fadhli



The whole process is automated and takes just a few minutes from code commit to production!

Example Alert (Alertmanager → Slack)



#alerts APP 10:42 AM

[ALERT] High CPU Usage
Application: my-flask-app
Environment: production
Metric: CPU Usage
Value: 92%
Threshold: > 80%
Time: 2026-07-17 10:42:00 UTC
View in Grafana: [Dashboard Link](#)

DevOps Best Practices Applied

- ✓ Infrastructure as Code (IaC) with Terraform
- ✓ CI/CD automation with GitLab CI
- ✓ Containerization with Docker
- ✓ Immutable infrastructure and easy rollbacks
- ✓ Monitoring, logging and alerting
- ✓ Security integrated in every stage (DevSecOps)
- ✓ Separation of concerns and least privilege access
- ✓ Versioning, traceability and documentation

Why this matters ?

- Faster time to market
- Fewer manual errors
- Better reliability
- Easy to scale
- Quick recovery in case of issues
- Full visibility from code to production

Failure Scenario & Recovery

⚠ Scenario: New deployment causes high error rate.

Recovery Steps:

- 1 Alert triggered by Prometheus (error rate > 5%).
- 2 Alertmanager sends notification to Slack and Email.
- 3 DevOps team checks Grafana dashboard and logs (Loki).
- 4 Rollback to previous Docker image in ECS.
- 5 Verify service health (health checks & metrics).
- 6 Post-mortem and fix the issue.

✓ Downtime: ~ 2 minutes ⌚ Impact: Minimal

Lessons Learned

- ★ Automate as much as possible, but test everything.
- ★ Monitoring & alerting are critical for quick detection.
- ★ Keep infrastructure simple and well-documented.
- ★ Security should be part of the pipeline, not an option.
- ★ Small, frequent releases are safer than big ones.
- ★ Always plan for failure and practice rollbacks.

“ DevOps is not a tool, it's a culture of collaboration, automation and continuous improvement. ”

Project Summary

</> Application: Python Flask API
 Container: Docker
 CI/CD: GitLab CI
 Infrastructure: AWS (Terraform)
 Monitoring: Prometheus, Grafana
 Logging: Loki
 Alerts: Alertmanager + Slack + Email

Total time from code to production:
~ 5 to 10 minutes